RESUMEN TEMA 1

RODRIGO SANAGUSTÍN SÁNCHEZ

ALGORITMOS

Los algoritmos son una serie de instrucciones que nos permiten resolver cualquier problema como una operación matemática o cualquier problema al que nos enfrentemos en la vida cotidiana.

LENGUAJES DE PROGRAMACIÓN

Los lenguajes de programación son idiomas creados para que un ser humano sea capaz de entenderlo fácilmente y pueda ser interpretado por una máquina. Este sigue una serie de normas y órdenes, al igual que cualquier lengua para comunicarse, y cada una de las órdenes realiza una tarea concreta y única. Existen varios tipos de lenguajes de programación, cada uno especializado en campos distintos. Por ejemplo, unos están más especializados para los cálculos matemáticos mientras otros están más especializados para hacer un videojuego. Ejemplos de lenguaje son Java, C, etc...

Todo programa necesita compilar el lenguaje de programación usado, para luego poder ejecutarse en un ordenador, por lo que tiene un tiempo de compilación (lo que tarda en compilar) y un tiempo de ejecución (lo que se tarda en ejecutar).

LENGUAJES MULTIPLATAFORMA

Los lenguajes multiplataforma son lenguajes que han sido creados para que, tras la compilación, el programa pueda ejecutarse en cualquier ordenador que tenga descargado la Máquina Virtual correspondiente. Estos lenguajes, como el Java, se crearon para facilitar la complejidad de lenguajes como C, los cuales necesitan que un mismo programa se compile para cada tipo de máquina. De ese modo, solo necesita compilarse una vez y se puede ejecutar en multitud de dispositivos.

NETBEANS IDE

NetBeans es un editor de texto donde se pueden escribir instrucciones que puedan ser interpretadas por la máquina virtual de Java. Esta aplicación es usada para crear y ejecutar programas, normalmente programas sencillos como crear aplicaciones para calcular precios de entradas, etc.

PALABRAS RESERVADAS

En Java, al igual que en otros tipos de lenguaje de programación, existen palabras con significados especiales que caracterizan al lenguaje. A continuación, se muestra una lista que incluyen una serie de palabras reservadas:

abstract	continue	float	native	strictfp	void
assert	default	for	new	super	volatile
boolean	do	if	package	switch	while
break	double	implements	private	synchronized	yield
byte	else	import	protected	this	
case	enum	instanceof	public	throw	
catch	extends	int	return	throws	
char	final	interface	short	transient	
class	finally	long	static	try	

Además, a esta lista de palabras hay

que añadir los valores *true*, *false* y *null*.

CONCEPTO DE VARIABLE

Una variable es el valor que puede tener un valor cualquiera de los comprendidos en un conjunto. Es por ello que este valor puede ir cambiando durante la ejecución de un programa. Las variables se asignan mediante el operador (=). La asignación de cada variable se denomina "identificador". En Java, dado que distingue entre mayúsculas y minúsculas, existen muchos identificadores diferentes de una misma palabra con simplemente variar una mayúscula o una minúscula de una variable. Como regla general, se usa la notación *Camel*, la cual consiste en escribir en mayúscula la primera letra de cada palabra para que se asemeje a las jorobas de un camello. Ej: *numEntradasVendidas*.

TIPOS PRIMITIVOS

En un programa, las variables se almacenan en la memoria de un ordenador, la cual se mide en bytes (ocho bits). Es por ello que usar un tamaño demasiado grande para una variable desaprovecharía la memoria, mientras que usar un espacio muy pequeño para una variable muy grande podría hacer que no quepa. Para ello, existen varios tipos de variables distintos, con distintos tamaños y rangos para todo tipo de situaciones. A continuación, se muestra una lista que incluyen los tipos primitivos en Java:

Tipo	Uso	Tamaño	Rango
`byte`	entero corto	8 bits	de –128 a 127
`short`	entero	16 bits	de –32 768 a 32 767
`int`	entero	32 bits	de –2 147 483 648 a 2 147 483 647
`long`	entero largo	64 bits	± 9 223 372 036 854 775 808
`float`	real precisión sencilla	32 bits	de -10^{32} a 10^{32}
`double`	real precisión doble	64 bits	de -10^{300} a 10^{300}
`boolean`	lógico	1 bit	`true` o `false`
`char`	texto	16 bits	cualquier carácter

VARIABLES DE OBJETO

Al igual que se pueden declarar variables de tipo primitivo, se pueden declarar clases como variables, los cuales son tipos de datos complejos denominadas *variables de objeto*.

CONSTANTES

Anteriormente se mencionó que las variables pueden cambiar a lo largo del programa. No obstante, las constantes son distintas, dado que una vez asignadas no cambian su valor durante el resto del programa. Para declarar una constante, se añade la palabra *final*.

Ej*: final int precioEntrada*;

COMENTARIOS

Un comentario son notas o comentarios que se usan para escribir la funcionalidad del código y facilitar la comprensión de la solución. Estos no son leídos por la consola al ejecutar el programa. En Java, existen tres tipos de comentario:

- Comentario multilínea: comentario que puede extenderse varias líneas, desde la apertura hasta el cierre. */* (…) **/*
- Comentario hasta final de línea: comentario que solo se extiende una línea, entre el carácter // y el final de línea. *// (…)*
- Comentario de documentación: es similar al comentario multilínea, solo que se inicia de manera distinta. */***

API DE JAVA

Son un conjunto de herramientas ya implementadas en Java que facilitan la labor del programador al ahorrar tiempo y trabajo, además de seguridad dado que funcionan bien porque han sido programadas y comprobadas por expertos. A estas se les denomina clases y facilitan multitud de tareas, como cálculos complejos, lectura o escritura de datos y el manejo de errores.

Para facilitar su organización, estas clases de la API se agrupan en paquetes, los cuales a su vez pueden agruparse en otros paquetes. Como ejemplos de paquetes encontramos *Math*, el cual proporciona herramientas para cálculos matemáticos.

Además de resolver operaciones matemáticas, la API permite mostrar mensajes en pantalla. Para ello, se usan los siguientes dos métodos mediante la clase System:

- *System.out.print ("Hola")*, que mostrará el mensaje literalmente en el monitor.
- *System.out.println ("Hola")*, que hará lo mismo que el anterior, pero añadirá una nueva línea (retorno de carro).

Incluso, la API no solo permite la salida por consola, sino que también permite la entrada de datos, es decir, permite leer lo que un usuario teclee, y esta respuesta sea leída por la consola. Para ello, necesitamos la clase Scanner, la cual se deberá implementar antes del algoritmo (debajo de package) mediante *import java.util.Scanner*, y en el algoritmo, se deberá configurar un nuevo escáner, mediante *Scanner sc = new Scanner(System.in)*. Una vez creado el escáner, podemos programar que se introduzcan datos para posteriormente ser leídos por la consola mediante *variable = sc.nextInt()* (en Int, se deberá poner el tipo de variable empleada en *variable*).

OPERACIONES BÁSICAS

En Java se encuentran una gran variedad de operadores para expresar diferentes datos. Existen cuatro tipos de operadores: Operador de asignación, Operadores aritméticos, Operadores relacionales, Operadores lógicos, Operadores opera y asigna y el Operador ternario:

- Operador de asignación (=): se usa para modificar el valor de una variable.
- Operadores aritméticos: se usan para hacer operaciones matemáticas sencillas.

Símbolo	Descripción
+	Suma
+	Más unario: positivo
–	Resta
–	Menor unario: negativo
*	Multiplicación
/	División
%	Módulo
++	Incremento en 1
– –	Decremento en 1

  - El operador *%* devuelve el resto de dividir dos números: 7%3 devolvería 1
  - Los operadores *++* y – se utilizan para incrementar o decrementar una variable en 1: a++ es lo mismo que a = a+1.
- Operadores relacionales: se usan para producir resultados lógicos o booleano a partir de comparaciones numéricas y su resultado solo permite dos posibles valores, que son *true* or *false*.

Símbolo	Descripción
==	Igual que
!=	Distinto que
<	Menor que
<=	Menor o igual que
>	Mayor que
>=	Mayor o igual que

- Operadores lógicos: se usan a partir de expresiones lógicas para devolver un valor lógico.

Símbolo	Descripción
&&	Operador and: Y
||	Operador or: O
!	Operador not: negación

- Operadores opera y asigna: se usan para realizar la operación indicada tomando como operandos el valor de la variable a la izquierda y a la derecha.
  Ej: var += 3 es lo mismo que var = var + 3.

Símbolo	Descripción
+=	Suma y asigna
-=	Resta y asigna
*=	Multiplica y asigna
/=	Divide y asigna
%=	Módulo y asigna

  Como son matemáticas, los operadores actuarán con orden de precedencia, por lo que no poner un paréntesis podría cambiar la operación al completo.
- Operador ternario: se usa para devolver un valor entre dos posibles. Se usa mediante:
  expresiónCondicional ? valor 1 : valor 2
  Si la expresión resulta cierta, se devolverá valor 1. Si resulta falsa, se devolverá valor 2.

RESUMEN TEMA 2

OPERADORES LÓGICOS:

Símbolo	Descripción
&&	Operador Y
||	Operador O
!	Operador negación

- Operador && será cierto si ambos operandos son ciertos:

a	b	a && b
falso	falso	falso
cierto	falso	falso
falso	cierto	falso
cierto	cierto	cierto

- Operador || será cierto si cualquiera de los operandos es cierto:

a	b	a || b
falso	falso	falso
cierto	falso	cierto
falso	cierto	cierto
cierto	cierto	cierto

- Operador ! cambia el valor al que se aplica, convirtiendo true en false y viceversa:

a	!a
falso	cierto
cierto	falso

CONDICIONAL SIMPLE: IF

El condicional *If* proporciona un control sobre una serie de acciones que pueden ejecutarse o no dependiendo de la evaluación de una condición, es decir, *if* se ejecutará si la condición es *true* y no se ejecutará cuando la condición sea *false*.

CONDICIONAL DOBLE: IF-ELSE

El condicional *if-else* funciona de forma similar al condicional *if*, lo que pasa que aquí se dividen en dos bloques de instrucciones. El primer bloque (*if*) se ejecutará cuando la condición sea *true* y el segundo bloque (*else*) se ejecutará cuando la condición sea *false*, ejecutando solo una de ellos.

Sintaxis: if (condición) {

} else {

}

ANIDACIÓN DE CONDICIONALES

Para realizar múltiples comprobaciones se pueden anidar tantos *if* o *if-else* como se necesiten.

Sintaxis *if-else*:

```
if (condición) {
} else if (condición 2) {
} else if (condición 3) {
} else if (condición 4) {
} else (condición 5) {
}
```

CONDICIONAL MÚLTIPLE: SWITCH

Es un condicional usado al necesitar utilizar muchos *if*/*if-else* anidados dado que estos resultan en muchas ocasiones difíciles de entender. Para ello se usa *switch*:

Ej:

```
switch (expresión) {
case valor1:
    conjunto instrucción 1
case valor2:
    conjunto instrucción 2
…
case valorN:
    conjunto instrucción N
default:
    conjunto instrucción default
}
```

El *switch* ejecuta desde el case verdadero hasta el final. Es decir, si el case *valor2* es el verdadero, se ejecutarán todos los siguientes case hasta el final (sin incluir el default, dado que este solo se ejecuta si ningún case es verdadero). Para que no ocurra esto, hay que añadir *break* al finalizar un bloque de instrucción para solo mostrar el valor de ese case. *break* salta hasta el } final.

UNICODE Y ASCII

Son dos sistemas de codificación de caracteres utilizados en la informática para representar texto. Aunque comparten utilidad, estos tienen diferencias entre sí.

ASCII (American Standard Code for Information Interchange) usa 7 bits, lo que permite codificar tan solo 128 caracteres. Fue creado solo para soportar caracteres en inglés, es decir, mayúsculas y minúsculas de cada letra, números del 0 al 9, y símbolos de puntuación.

Al ser solo para caracteres ingleses, es por ello que se creó UNICODE, un sistema que usa hasta 32 bits, lo que permite codificar hasta 1 millón de caracteres. Este incluye caracteres de prácticamente todos los países del mundo, incluyendo emojis y otros objetos de expresión que se usan diariamente. Este puede ser codificado en varias formas:

- UTF 8: usa entre 1 y 4 bytes por carácter y es compatible con ASCII.
- UTF 16: usa entre 2 y 4 bytes por carácter.
- UTF 32: usa 4 bytes por carácter.

Aunque presentan diferencias, ambos sistemas presentan compatibilidad, es decir, los primeros 128 caracteres de UNICODE son exactamente los 128 caracteres de ASCII, y los siguientes son expansiones que se han ido creando con el paso del tiempo para crear un sistema de codificación de caracteres universal. Es por ello que ASCII se considera un subconjunto de UNICODE, y este último es el sistema estándar utilizado en la web y la informática.

TRABAJAR CON LA COMA DECIMAL

Cuando trabajamos con números decimales en Java, debemos saber lo siguiente: el código Java siempre usa el punto (.) para decimales. En cambio, un usuario puede usar la coma (,) según su región, como es el caso de España. Para ello, existe una solución:

Para ello se usa *Locale*, el cual representa una combinación de idioma y región y describe un contexto cultural y lingüístico. Para poder trabajar con *Locale* tenemos que crearlo mediante el siguiente comando: *Locale espana = Locale.of('es', 'ES')*. Al no haber Locale.SPAIN en todas las versiones, es recomendable usar *Locale.of('es', 'ES')*. No obstante, este cambio solo afectará en el visual, dado que para Java el dato siempre se almacena con un punto (.), y solo cambia la forma en mostrar el texto para que el usuario lo vea. Para el texto de entrada, deberemos configurar el escáner para la región local para poder usar la coma en operaciones y números. En el caso de España, se usa lo siguiente: *Locale conf_regional = Locale.of("es", "ES")*

Para mostrar la coma (,) de un número por salida de consola no basta con poner el comando *System.out.println(…)* dado que te lo imprimirá con un punto (.). Para ello, necesitamos usar *printf*, para que se pueda mostrar la coma en la salida de la consola:

System.out.printf(espana, "Precio: %.2f%n", precio);

Donde:

- *%.2f*: Número flotante con 2 decimales.
- *%n*: Salto de línea independiente del sistema operativo.